INSTITUTE OF ENGINEERING & MANAGEMENT

Good Education, Good Jobs

Course Name: Advanced Programming (Object Oriented Programming) Laboratory

Project Title: Smart Food Delivery Order Assignment System

Student Name: Aishika Das

Roll Number: 23

Instructor Name: Prof. Soumendu Biswas, Dr. Susovan Jana, Prof. Swagatam Basu

Spring 2026

1. Problem Statement:

Food delivery platforms must efficiently assign incoming orders to available delivery partners. Inefficient assignment can lead to delayed deliveries, increased delivery time, and poor customer experience. In many situations, delivery partners are assigned manually or through simplistic rules that do not consider factors such as distance, availability, and delivery priority. This project proposes a Smart Food Delivery Order Assignment System that models orders, delivery partners, and assignment logic through a structured object-oriented design. The system will dynamically assign orders to the most suitable delivery partner based on predefined criteria such as partner availability and estimated distance. The architecture emphasizes modular design and extensibility, allowing different assignment strategies to be implemented through polymorphism. The system demonstrates core object-oriented principles while simulating a realistic logistics coordination problem.

2. System Design

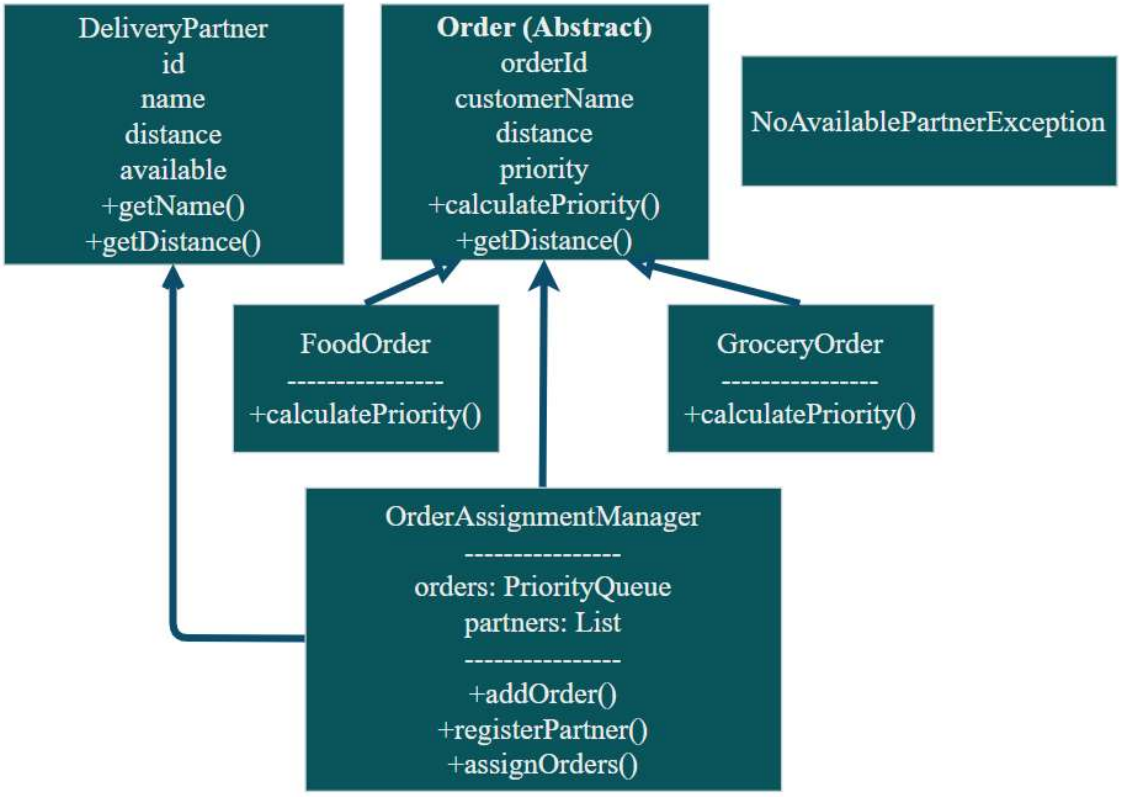


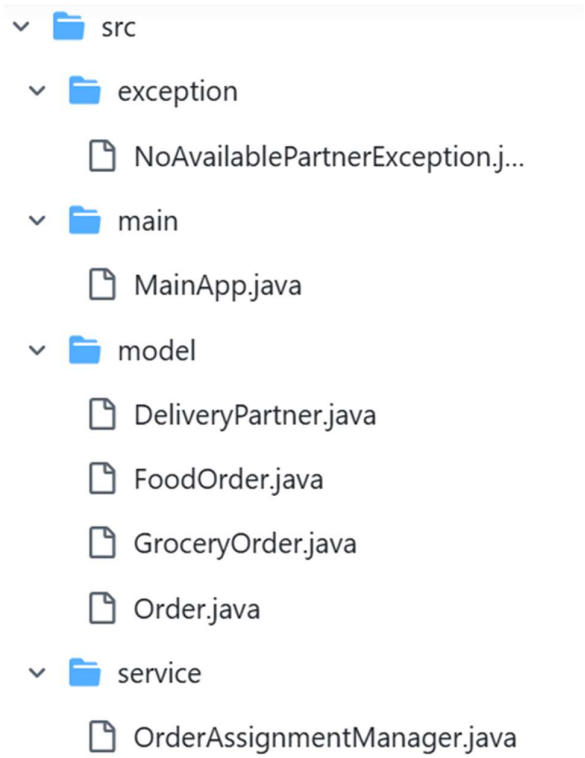
Fig 1. UML Class Diagram

Description of Major Classes

- Order (Abstract Class): Represents a generic order with attributes like ID, customer name, distance, and priority.
- FoodOrder / GroceryOrder: Extend Order and implement different priority calculation strategies.

- DeliveryPartner: Represents delivery personnel with attributes like ID, name, availability, and distance.
- OrderAssignmentManager: Core logic class responsible for managing orders, partners, and assignment strategy.
- NoAvailablePartnerException: Handles situations when no delivery partner is available.
- MainApp: Acts as the entry point providing menu-driven interaction.

Package Structure



Interaction Flow

- User adds delivery partners
- User adds orders
- Orders stored in PriorityQueue
- System finds nearest available partner
- Assigns order
- If all busy → waits
- On completion → assignment resumes

3. OOP Concepts Demonstrated

Abstraction

- **Where:** Order class
- **Why:** To define common structure for all orders
- **Code:**
- ```
  public abstract class Order {
      protected int orderId;
      protected double distance;

      public abstract int calculatePriority();
  }
  ```
- **Explanation:**
  Abstract class ensures all order types implement their own priority logic.

Inheritance

- **Where:** FoodOrder, GroceryOrder
- **Why:** To reuse common properties of Order
- **Code:**
- ```
  public class FoodOrder extends Order {
      public int calculatePriority() {
          return (int)(100 - distance);
      }
  }
  ```
- **Explanation:**
  Child classes extend Order and override behavior.

Polymorphism

- **Where:** calculatePriority()
- **Why:** Different order types behave differently
- **Code:**
- ```
  Order o = new FoodOrder(...);
  o.calculatePriority();
  ```
- **Explanation:**
  Same method behaves differently depending on object type.

Encapsulation

- **Where:** Private fields + getters
- **Why:** To protect data
- **Code:**
- ```
  public double getDistance() {
      return distance;
  }
  ```
- **Explanation:**
  Fields are accessed through methods, not directly.

Exception Handling

- **Where:** Assignment logic
- **Why:** Handle no available partner
- **Code:**
- ```
  if (partners.isEmpty()) {
      throw new NoAvailablePartnerException("No delivery partners");
  }
  ```
- **Explanation:**
  Prevents system crash and handles edge cases.

1. <u>Implementation Highlights</u>

1. Priority Queue for Orders

- `orders = new PriorityQueue<>((a, b) -> b.getPriority() - a.getPriority());`
- **Justification:**
  Ensures high-priority orders are processed first.

2. Nearest Partner Selection

- `double diff = Math.abs(p.getDistance() - o.getDistance());`
- **Justification:**
  Assigns the closest delivery partner to reduce delivery time.

3. Availability Handling

- ```
  if (nearest == null) {
      System.out.println("All delivery partners are busy.");
  }
  ```
- **Justification:**
  Simulates real-world limitation of resources.

4. Round-Robin / Smart Assignment

- `nearest.setAvailable(false);`
- **Justification:**
  Ensures partner is not assigned multiple orders simultaneously.

5. Interactive Menu System

- `int choice = sc.nextInt();`
- **Justification:**
  Allows dynamic user input and testing.

6. Testing & Error Handling

Test Cases Considered:

1. Basic Assignment Test:
   - Input: 2 delivery partners, 2 orders
   - Expected: Each order assigned to nearest partner
2. Multiple Orders Test
   - Input: 2 partners, 5 orders
   - Expected: Only 2 orders assigned, rest wait
3. No Partner Test
   - Input: Orders added without partners
   - Expected: Exception thrown
4. Dynamic Input Test
   - Input: Add partners and orders at runtime
   - Expected: System handles input correctly

Edge Cases Handled:

- ✓ No delivery partners available
- ✓ No orders in system
- ✓ All partners busy
- ✓ Multiple orders with same distance
- ✓ Invalid menu choice

Failure Scenarios:

Scenario	Handling
No partner available	Custom exception thrown
Empty order queue	Message displayed
Invalid input	Default case handled
All partners busy	System pauses assignment

7. Git Workflow

Screenshot of Commit History

order class was created
AishikaTheExplorer committed 13 hours ago

Food order class was created
AishikaTheExplorer committed 13 hours ago

grocery order class was created
AishikaTheExplorer committed 13 hours ago

delivery partner class was added
AishikaTheExplorer committed 13 hours ago

when no delivery person available,that case was handled
AishikaTheExplorer committed 13 hours ago

main class was added for execution
AishikaTheExplorer committed 13 hours ago

The order assignment task was added as a class
AishikaTheExplorer committed 13 hours ago

user input has been added
AishikaTheExplorer committed 11 hours ago

trying to fix the error due to wrong logic for assigning delivery par...
...tner
AishikaTheExplorer committed 11 hours ago

trying to maike the nearest delivery person get the order and trying ...
...to make the algorithm even better
AishikaTheExplorer committed 11 hours ago

priority queue embedded and it is working correctly
AishikaTheExplorer committed 10 hours ago

Explanation of Development Progression

The project was developed incrementally using version control. Each major feature was implemented in separate commits to maintain clarity and traceability.

Development Timeline:

1. Initial Setup
   - Created project structure
   - Added package hierarchy
2. Core Model Classes
   - Implemented Order, FoodOrder, GroceryOrder
   - Established inheritance
3. Delivery Partner Module
   - Added DeliveryPartner class
   - Implemented availability logic
4. Assignment Logic
   - Implemented OrderAssignmentManager
   - Added priority queue
5. Advanced Features
   - Nearest partner assignment
   - Availability constraint handling
6. User Interface
   - Menu-driven input system
7. Refactoring & Debugging
   - Fixed encapsulation issues
   - Added getters and improved output

Git Best Practices Followed

✓ Multiple meaningful commits

✓ Feature-based commits

✓ Clear commit messages

✓ Incremental development

8. Conclusion & Future Scope

The Smart Food Delivery Order Assignment System successfully demonstrates the application of object-oriented design principles in solving a real-world logistics problem. The system efficiently assigns delivery partners to orders based on distance and availability while handling constraints such as limited resources. The use of abstraction, inheritance, polymorphism, and encapsulation ensures modularity and maintainability. Additionally, the system incorporates dynamic user input and queue-based scheduling to simulate realistic delivery operations.

Future Scope

- Real-time GPS Integration: Use live location instead of static distance
- Graph-based Routing Algorithm: Use shortest path algorithms like Dijkstra
- Database Integration: Store orders and partner data persistently
- GUI Interface: Replace console with graphical interface
- Machine Learning Optimization: Predict delivery time and optimize assignments

APPENDIX:

Initial project proposal:

Smart Food Delivery Order Assignment System

Problem Statement:

Food delivery platforms must efficiently assign incoming orders to available delivery partners. Inefficient assignment can lead to delayed deliveries, increased delivery time, and poor customer experience. In many situations, delivery partners are assigned manually or through simplistic rules that do not consider factors such as distance, availability, and delivery priority. This project proposes a Smart Food Delivery Order Assignment System that models orders, delivery partners, and assignment logic through a structured object-oriented design. The system will dynamically assign orders to the most suitable delivery partner based on predefined criteria such as partner availability and estimated distance. The architecture emphasizes modular design and extensibility, allowing different assignment strategies to be implemented through polymorphism. The system demonstrates core object-oriented principles while simulating a realistic logistics coordination problem.

Target User:

Food delivery service administrators managing delivery assignments and delivery partners in small-scale food delivery operations.

Core Features:

- Create and manage food delivery orders
- Register and manage delivery partners
- Automatic order assignment to available delivery partners
- Priority-based assignment logic
- Delivery status tracking (assigned, in-progress, completed)
- File-based persistence for orders and delivery records
- Exception handling for invalid operations and unavailable partners

OOP Concepts to be Used:

- Abstraction: An abstract Order class representing common order attributes and behaviors.
- Inheritance: Specialized subclasses such as FoodOrder and GroceryOrder extending the base order structure.
- Polymorphism: Different assignment strategies or priority calculations implemented through overridden methods.
- Exception Handling: Custom exceptions such as NoAvailableDeliveryPartnerException and InvalidOrderException.

Collections / File Handling:

- PriorityQueue for order assignment
- ArrayList / HashMap for managing orders and delivery partners
- File handling for saving and retrieving system data

Proposed Architecture Description:

The system follows a modular object-oriented architecture where core domain entities such as Order and DeliveryPartner encapsulate delivery-related data and behavior. Specialized order types extend the abstract order class to support different delivery scenarios. A central OrderAssignmentManager coordinates order processing and assigns orders to delivery partners using a priority-based assignment strategy. Collections are used to efficiently manage orders and delivery partners, while a persistence layer handles file storage for system data. This architecture separates domain logic, assignment logic, and data management, ensuring clarity, extensibility, and maintainability.

Review mail received:

S soumadip.biswas@ieminternational.ai

To: Aishika Das

Tue 3/10/2026 10:09 AM

Flagged

Dear Aishika Das,

Project: Smart Food Delivery Order Assignment System
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal clearly articulates a real-world problem in food delivery and proposes a well-structured solution. The features are comprehensive and appropriate for demonstrating core OOP principles. The explicit mention and application of Abstraction, Inheritance, and Polymorphism, particularly for different assignment strategies, are very strong. The modular architecture with clear separation of concerns (domain, assignment, persistence) is well-thought-out and promotes extensibility and maintainability. The scope is realistic for a university project.

Improvements:
While encapsulation is clearly implied in the architecture section ('Order and DeliveryPartner encapsulate...'), explicitly listing it among the OOP Concepts would provide a more complete picture of the fundamental principles being applied. The proposal could briefly elaborate on the specific types of 'Specialized order types' and 'different delivery scenarios' that extend the abstract order class, providing more concrete examples of polymorphism in action beyond just assignment strategies. A brief mention of the intended user interface (e.g., console-based) would also add clarity.

Regards,
Course Instructor